

# CRYPTOGRAPHY AND NETWORK SECURITY

Course Code: **23CS402** • Semester: **4-1** • Comprehensive 5-Unit Academic Compendium

---

**Course Objective & Overview:** Exhaustive textbook on modern cryptographic primitives, symmetric and asymmetric ciphers, number theory proofs, digital signatures, hash functions, and network security protocols.

# Unit 1: Security Goals, Threat Models & Classical Encryption

## 1.1 CIA Security Triad, Attacks & Security Architecture

Computer security ensures Confidentiality (preventing unauthorized disclosure), Integrity (preventing unauthorized modification), and Availability (ensuring timely authorized access). Threats are categorized into Passive Attacks (eavesdropping, traffic analysis) and Active Attacks (masquerade, replay, message tampering, denial of service).

## 1.2 Classical Substitution & Transposition Ciphers

Monoalphabetic ciphers (Caesar, Playfair) substitute characters but are vulnerable to frequency analysis. Polyalphabetic ciphers (Vigenere, Hill Cipher) use modular matrix transformations  $C = K P \bmod 26$ . Transposition ciphers (Rail Fence, Columnar Transposition) rearrange character positions to maximize diffusion.

## 1.3 Modular Arithmetic & Euclidean Algorithms

Modular arithmetic forms the algebraic basis of modern cryptography over Galois Fields. The Euclidean Algorithm computes  $\gcd(a, b)$ . The Extended Euclidean Algorithm calculates the modular multiplicative inverse  $a^{-1} \bmod m$  such that  $(a * a^{-1}) = 1 \bmod m$ , existing iff  $\gcd(a, m) = 1$ .

## 1.4 Number Theory Principles & Euler's Totient Function

Fermat's Little Theorem states  $a^{p-1} = 1 \bmod p$  for prime  $p$  and  $\gcd(a, p) = 1$ . Euler's Totient Function  $\phi(n)$  counts integers  $k < n$  coprime to  $n$ ; for semiprime  $n = p * q$ ,  $\phi(n) = (p - 1)(q - 1)$ . Euler's Theorem generalizes:  $a^{\phi(n)} = 1 \bmod n$ .

## University Examination Review Questions — Unit 1

### Q1. Explain the fundamental theoretical and practical significance of Security Goals, Threat Models & Classical Encryption.

Solution: Security Goals, Threat Models & Classical Encryption provides the structural foundation required for engineering implementation. It decomposes complex computational tasks into rigorous mathematical models and modular subsystems, guaranteeing deterministic performance and predictable resource utilization.

### Q2. What are the key architectural constraints and design tradeoffs associated with Security Goals, Threat Models & Classical Encryption?

Solution: Primary tradeoffs include time complexity vs space complexity, throughput vs latency, and hardware abstraction vs direct memory manipulation. Proper selection depends on specific latency budgets and deployment environments.

**Q3. Derive or explain the step-by-step mathematical/algorithmic procedure for core operations in Security Goals, Threat Models & Classical Encryption.**

Solution: 1. Input state initialization and boundary parameter validation. 2. Iterative or recursive state transitions preserving invariant constraints. 3. Convergence termination check. 4. Output verification and error propagation handling.

**Q4. Compare alternative methodologies and explain when each approach is preferred.**

Solution: Deterministic approaches guarantee bounded worst-case bounds; heuristic/probabilistic methods provide near-optimal results for NP-hard domains in linear or sub-quadratic time.

**Q5. Analyze the worst-case, best-case, and average-case performance bounds.**

Solution: Standard operations exhibit  $O(1)$  or  $O(\log N)$  optimal lookup,  $O(N \log N)$  divide-and-conquer processing, and  $O(N^2)$  unmitigated worst-case bounds under adversarial inputs.

**Q6. Discuss memory management, cache locality, and runtime optimization strategies.**

Solution: Spatial locality maximizes CPU L1/L2 cache hit ratios through contiguous memory layout; temporal locality reuses recently referenced data structures before eviction.

**Q7. Describe error detection, exception handling, and fault tolerance mechanisms.**

Solution: Robust implementations employ defensive bounds checking, transactional rollback semantics, write-ahead logging, and grace degradation paths.

**Q8. How does modern industry software integrate these concepts at enterprise scale?**

Solution: Scaled deployments utilize multi-threaded asynchronous workers, microservice partitioning, distributed consensus protocols, and hardware vectorization (SIMD / GPU acceleration).

**Q9. What are common implementation pitfalls and how are they mitigated?**

Solution: Common pitfalls include off-by-one boundary errors, unhandled null references, race conditions, memory leaks, and premature optimization without profiling.

**Q10. Summarize the key formulas, theorems, and critical revision points for Unit 1.**

Solution: Master the fundamental governing equations, invariant properties, state machine transitions, and asymptotic space/time complexity bounds documented in this chapter.

## Unit 2: Symmetric Block Ciphers: DES & AES Standards

### 2.1 Feistel Cipher Network & DES Architecture

Data Encryption Standard (DES) executes 16 Feistel rounds over 64-bit plaintext blocks using 56-bit keys. Feistel networks provide reversible decryption using the identical round function by reversing key schedules. Triple DES (3DES) achieves 112/168-bit security via EDE (Encrypt-Decrypt-Encrypt) chaining.

### 2.2 Advanced Encryption Standard (AES) Mathematical Operations

AES is an iterated Substitution-Permutation Network operating on a 4x4 state matrix of bytes over finite field  $GF(2^8)$ . AES-128 (10 rounds), AES-192 (12 rounds), and AES-256 (14 rounds) execute 4 distinct transformations: SubBytes (non-linear S-Box), ShiftRows (cyclical byte shifting), MixColumns (matrix polynomial multiplication), and AddRoundKey (XOR with round subkeys).

### 2.3 Block Cipher Modes of Operation

1. Electronic Codebook (ECB; deterministic, leaks patterns), 2. Cipher Block Chaining (CBC; XORs plaintext with previous ciphertext block using an IV), 3. Cipher Feedback (CFB) & Output Feedback (OFB), 4. Counter Mode (CTR; transforms block cipher into stream cipher, enabling parallel high-speed hardware encryption).

### 2.4 Differential & Linear Cryptanalysis Resistance

AES S-Boxes are mathematically constructed using multiplicative inverses in  $GF(2^8)$  combined with affine transformations, provably maximizing non-linearity and immunity against linear and differential cryptanalysis attacks.

## University Examination Review Questions — Unit 2

### Q1. Explain the fundamental theoretical and practical significance of Symmetric Block Ciphers: DES & AES Standards.

Solution: Symmetric Block Ciphers: DES & AES Standards provides the structural foundation required for engineering implementation. It decomposes complex computational tasks into rigorous mathematical models and modular subsystems, guaranteeing deterministic performance and predictable resource utilization.

### Q2. What are the key architectural constraints and design tradeoffs associated with Symmetric Block Ciphers: DES & AES Standards?

Solution: Primary tradeoffs include time complexity vs space complexity, throughput vs latency, and hardware abstraction vs direct memory manipulation. Proper selection depends on specific latency budgets and deployment environments.

### Q3. Derive or explain the step-by-step mathematical/algorithmic procedure for core operations in Symmetric Block Ciphers: DES & AES Standards.

Solution: 1. Input state initialization and boundary parameter validation. 2. Iterative or recursive state transitions preserving invariant constraints. 3. Convergence termination check. 4. Output verification and error propagation handling.

**Q4. Compare alternative methodologies and explain when each approach is preferred.**

Solution: Deterministic approaches guarantee bounded worst-case bounds; heuristic/probabilistic methods provide near-optimal results for NP-hard domains in linear or sub-quadratic time.

**Q5. Analyze the worst-case, best-case, and average-case performance bounds.**

Solution: Standard operations exhibit  $O(1)$  or  $O(\log N)$  optimal lookup,  $O(N \log N)$  divide-and-conquer processing, and  $O(N^2)$  unmitigated worst-case bounds under adversarial inputs.

**Q6. Discuss memory management, cache locality, and runtime optimization strategies.**

Solution: Spatial locality maximizes CPU L1/L2 cache hit ratios through contiguous memory layout; temporal locality reuses recently referenced data structures before eviction.

**Q7. Describe error detection, exception handling, and fault tolerance mechanisms.**

Solution: Robust implementations employ defensive bounds checking, transactional rollback semantics, write-ahead logging, and graceful degradation paths.

**Q8. How does modern industry software integrate these concepts at enterprise scale?**

Solution: Scaled deployments utilize multi-threaded asynchronous workers, microservice partitioning, distributed consensus protocols, and hardware vectorization (SIMD / GPU acceleration).

**Q9. What are common implementation pitfalls and how are they mitigated?**

Solution: Common pitfalls include off-by-one boundary errors, unhandled null references, race conditions, memory leaks, and premature optimization without profiling.

**Q10. Summarize the key formulas, theorems, and critical revision points for Unit 2.**

Solution: Master the fundamental governing equations, invariant properties, state machine transitions, and asymptotic space/time complexity bounds documented in this chapter.

## Unit 3: Asymmetric Public-Key Cryptography & Key Distribution

### 3.1 Public-Key Principles & Trapdoor One-Way Functions

Asymmetric cryptography utilizes key pairs: a Public Key for encryption/verification and a Private Key for decryption/signing, resolving symmetric key distribution bottlenecks via trapdoor one-way mathematical functions.

### 3.2 RSA Cryptosystem Algorithm & Security Proof

RSA key generation: Select large primes  $p$  and  $q$ , compute modulus  $n = p * q$  and  $\phi(n) = (p-1)(q-1)$ . Choose public exponent  $e$  coprime to  $\phi(n)$ . Compute private key  $d = e^{-1} \bmod \phi(n)$ . Encryption:  $C = M^e \bmod n$ . Decryption:  $M = C^d \bmod n$ . Security relies on the computational intractability of integer factorization.

### 3.3 Diffie-Hellman Key Exchange & MITM Attack Mitigation

Diffie-Hellman enables two parties to establish a shared secret over an insecure channel via modular exponentiation:  $K = (g^a \bmod p)^b \bmod p = g^{ab} \bmod p$ . Unauthenticated Diffie-Hellman is vulnerable to Man-In-The-Middle (MITM) attacks, resolved by signing exchanges with digital certificates.

### 3.4 Elliptic Curve Cryptography (ECC)

ECC defines algebraic groups over points  $(x, y)$  on Weierstrass cubic curves  $y^2 = x^3 + ax + b \bmod p$ . ECC delivers equivalent RSA-2048 security with compact 256-bit keys, drastically cutting compute cycles and bandwidth for mobile and IoT devices.

## University Examination Review Questions — Unit 3

### Q1. Explain the fundamental theoretical and practical significance of Asymmetric Public-Key Cryptography & Key Distribution.

Solution: Asymmetric Public-Key Cryptography & Key Distribution provides the structural foundation required for engineering implementation. It decomposes complex computational tasks into rigorous mathematical models and modular subsystems, guaranteeing deterministic performance and predictable resource utilization.

### Q2. What are the key architectural constraints and design tradeoffs associated with Asymmetric Public-Key Cryptography & Key Distribution?

Solution: Primary tradeoffs include time complexity vs space complexity, throughput vs latency, and hardware abstraction vs direct memory manipulation. Proper selection depends on specific latency budgets and deployment environments.

### Q3. Derive or explain the step-by-step mathematical/algorithmic procedure for core operations in Asymmetric Public-Key Cryptography & Key Distribution.

Solution: 1. Input state initialization and boundary parameter validation. 2. Iterative or recursive state transitions preserving invariant constraints. 3. Convergence termination check. 4. Output verification and error propagation handling.

**Q4. Compare alternative methodologies and explain when each approach is preferred.**

Solution: Deterministic approaches guarantee bounded worst-case bounds; heuristic/probabilistic methods provide near-optimal results for NP-hard domains in linear or sub-quadratic time.

**Q5. Analyze the worst-case, best-case, and average-case performance bounds.**

Solution: Standard operations exhibit  $O(1)$  or  $O(\log N)$  optimal lookup,  $O(N \log N)$  divide-and-conquer processing, and  $O(N^2)$  unmitigated worst-case bounds under adversarial inputs.

**Q6. Discuss memory management, cache locality, and runtime optimization strategies.**

Solution: Spatial locality maximizes CPU L1/L2 cache hit ratios through contiguous memory layout; temporal locality reuses recently referenced data structures before eviction.

**Q7. Describe error detection, exception handling, and fault tolerance mechanisms.**

Solution: Robust implementations employ defensive bounds checking, transactional rollback semantics, write-ahead logging, and graceful degradation paths.

**Q8. How does modern industry software integrate these concepts at enterprise scale?**

Solution: Scaled deployments utilize multi-threaded asynchronous workers, microservice partitioning, distributed consensus protocols, and hardware vectorization (SIMD / GPU acceleration).

**Q9. What are common implementation pitfalls and how are they mitigated?**

Solution: Common pitfalls include off-by-one boundary errors, unhandled null references, race conditions, memory leaks, and premature optimization without profiling.

**Q10. Summarize the key formulas, theorems, and critical revision points for Unit 3.**

Solution: Master the fundamental governing equations, invariant properties, state machine transitions, and asymptotic space/time complexity bounds documented in this chapter.

## Unit 4: Cryptographic Hash Functions & Digital Signatures

### 4.1 Hash Properties (Collision Resistance & Avalanche Effect)

Cryptographic hash functions  $H(M)$  produce fixed-size digests satisfying: 1. Pre-image Resistance (one-way), 2. Second Pre-image Resistance (weak collision resistance), 3. Collision Resistance (hard to find any  $M1 \neq M2$  with  $H(M1) = H(M2)$ ). Small input bit changes trigger widespread output divergence (Avalanche Effect).

### 4.2 SHA-256 / SHA-512 Architecture & HMAC

Secure Hash Algorithm 2 (SHA-256) processes 512-bit message blocks through 64 compression rounds using bitwise logical functions (Ch, Maj, Sigma). Hash-based Message Authentication Codes (HMAC) combine cryptographic hashes with secret shared keys for API packet authentication.

### 4.3 Digital Signatures & Non-Repudiation

A digital signature is generated by encrypting a message hash with the sender's private key:  $S = \text{Sign}_{\{K_{\text{priv}}\}}(H(M))$ . The recipient verifies using the sender's public key:  $\text{Verify}_{\{K_{\text{pub}}\}}(S) == H(M)$ , guaranteeing Authenticity, Integrity, and Non-Repudiation.

### 4.4 Public Key Infrastructure (PKI) & X.509 Certificates

PKI binds public keys to verified identities through trusted Certificate Authorities (CAs). X.509 certificates contain Subject Info, Public Key, Issuer Signature, Validity Period, and Key Usage constraints, authenticated via Certificate Revocation Lists (CRL) and OCSP stapling.

## University Examination Review Questions — Unit 4

**Q1. Explain the fundamental theoretical and practical significance of Cryptographic Hash Functions & Digital Signatures.**

Solution: Cryptographic Hash Functions & Digital Signatures provides the structural foundation required for engineering implementation. It decomposes complex computational tasks into rigorous mathematical models and modular subsystems, guaranteeing deterministic performance and predictable resource utilization.

**Q2. What are the key architectural constraints and design tradeoffs associated with Cryptographic Hash Functions & Digital Signatures?**

Solution: Primary tradeoffs include time complexity vs space complexity, throughput vs latency, and hardware abstraction vs direct memory manipulation. Proper selection depends on specific latency budgets and deployment environments.

**Q3. Derive or explain the step-by-step mathematical/algorithmic procedure for core operations in Cryptographic Hash Functions & Digital Signatures.**



Solution: 1. Input state initialization and boundary parameter validation. 2. Iterative or recursive state transitions preserving invariant constraints. 3. Convergence termination check. 4. Output verification and error propagation handling.

**Q4. Compare alternative methodologies and explain when each approach is preferred.**

Solution: Deterministic approaches guarantee bounded worst-case bounds; heuristic/probabilistic methods provide near-optimal results for NP-hard domains in linear or sub-quadratic time.

**Q5. Analyze the worst-case, best-case, and average-case performance bounds.**

Solution: Standard operations exhibit  $O(1)$  or  $O(\log N)$  optimal lookup,  $O(N \log N)$  divide-and-conquer processing, and  $O(N^2)$  unmitigated worst-case bounds under adversarial inputs.

**Q6. Discuss memory management, cache locality, and runtime optimization strategies.**

Solution: Spatial locality maximizes CPU L1/L2 cache hit ratios through contiguous memory layout; temporal locality reuses recently referenced data structures before eviction.

**Q7. Describe error detection, exception handling, and fault tolerance mechanisms.**

Solution: Robust implementations employ defensive bounds checking, transactional rollback semantics, write-ahead logging, and graceful degradation paths.

**Q8. How does modern industry software integrate these concepts at enterprise scale?**

Solution: Scaled deployments utilize multi-threaded asynchronous workers, microservice partitioning, distributed consensus protocols, and hardware vectorization (SIMD / GPU acceleration).

**Q9. What are common implementation pitfalls and how are they mitigated?**

Solution: Common pitfalls include off-by-one boundary errors, unhandled null references, race conditions, memory leaks, and premature optimization without profiling.

**Q10. Summarize the key formulas, theorems, and critical revision points for Unit 4.**

Solution: Master the fundamental governing equations, invariant properties, state machine transitions, and asymptotic space/time complexity bounds documented in this chapter.

## Unit 5: Network Security Protocols, Firewalls & System Defenses

### 5.1 IPsec Protocol Architecture (AH, ESP & IKE)

IP Security (IPsec) operates at the Network Layer. Authentication Header (AH) guarantees packet integrity; Encapsulating Security Payload (ESP) provides both confidentiality and authentication. Modes include Transport Mode (encrypts payload only) and Tunnel Mode (encapsulates entire IP packet inside VPN tunnels).

### 5.2 Transport Layer Security (TLS 1.3 Handshake)

TLS secures application protocols (HTTPS, SSH). TLS 1.3 achieves single-round-trip (1-RTT) or zero-round-trip (0-RTT) handshakes using ephemeral Diffie-Hellman key exchange (ECDHE) for Perfect Forward Secrecy (PFS), encrypting all traffic after the Initial Client Hello.

### 5.3 Firewalls, Stateful Inspection & Web Application Firewalls (WAF)

Firewalls filter network traffic: 1. Packet Filtering (inspects IP headers and ports), 2. Stateful Inspection (tracks TCP 3-way handshake and connection states), 3. Application Layer Gateways / WAF (inspects HTTP payloads to block SQL Injection, XSS, and CSRF attacks).

### 5.4 Intrusion Detection (IDS/IPS) & DDoS Mitigation

Intrusion Detection Systems utilize Signature-Based Detection (known vulnerability patterns) and Anomaly-Based Detection (statistical ML deviations). Distributed Denial of Service (DDoS) mitigation leverages Anycast routing, Rate Limiting, SYN Cookies, and Cloud Scrubbing centers.

## University Examination Review Questions — Unit 5

#### **Q1. Explain the fundamental theoretical and practical significance of Network Security Protocols, Firewalls & System Defenses.**

Solution: Network Security Protocols, Firewalls & System Defenses provides the structural foundation required for engineering implementation. It decomposes complex computational tasks into rigorous mathematical models and modular subsystems, guaranteeing deterministic performance and predictable resource utilization.

#### **Q2. What are the key architectural constraints and design tradeoffs associated with Network Security Protocols, Firewalls & System Defenses?**

Solution: Primary tradeoffs include time complexity vs space complexity, throughput vs latency, and hardware abstraction vs direct memory manipulation. Proper selection depends on specific latency budgets and deployment environments.

#### **Q3. Derive or explain the step-by-step mathematical/algorithmic procedure for core operations in Network Security Protocols, Firewalls & System Defenses.**

Solution: 1. Input state initialization and boundary parameter validation. 2. Iterative or recursive state transitions preserving invariant constraints. 3. Convergence termination check. 4. Output verification and error propagation handling.

**Q4. Compare alternative methodologies and explain when each approach is preferred.**

Solution: Deterministic approaches guarantee bounded worst-case bounds; heuristic/probabilistic methods provide near-optimal results for NP-hard domains in linear or sub-quadratic time.

**Q5. Analyze the worst-case, best-case, and average-case performance bounds.**

Solution: Standard operations exhibit  $O(1)$  or  $O(\log N)$  optimal lookup,  $O(N \log N)$  divide-and-conquer processing, and  $O(N^2)$  unmitigated worst-case bounds under adversarial inputs.

**Q6. Discuss memory management, cache locality, and runtime optimization strategies.**

Solution: Spatial locality maximizes CPU L1/L2 cache hit ratios through contiguous memory layout; temporal locality reuses recently referenced data structures before eviction.

**Q7. Describe error detection, exception handling, and fault tolerance mechanisms.**

Solution: Robust implementations employ defensive bounds checking, transactional rollback semantics, write-ahead logging, and graceful degradation paths.

**Q8. How does modern industry software integrate these concepts at enterprise scale?**

Solution: Scaled deployments utilize multi-threaded asynchronous workers, microservice partitioning, distributed consensus protocols, and hardware vectorization (SIMD / GPU acceleration).

**Q9. What are common implementation pitfalls and how are they mitigated?**

Solution: Common pitfalls include off-by-one boundary errors, unhandled null references, race conditions, memory leaks, and premature optimization without profiling.

**Q10. Summarize the key formulas, theorems, and critical revision points for Unit 5.**

Solution: Master the fundamental governing equations, invariant properties, state machine transitions, and asymptotic space/time complexity bounds documented in this chapter.